

Figure 1: Sample *gitk* output

Figure 2: Shell prompt showing the Git branch

distribution is older and you would like to install the latest version, you can obtain the source from <https://git-scm.com/>. This may be the only option in case you do not have administrator access to install packages, yet want to install Git in your home directory.

Once done, you need to fill in information about yourself in Git so that it can be used in the commits made by you. This step is mandatory for every developer. So proceed as follows:

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your.name@yourcompany.com"
```

Support tools

Along with Git, it is useful to install a graphical tool that displays the history of a Git repository. This tool is called *gitk*. If this is not already on your system, it can be installed via the *gitk* package.

A sample output of the tool is shown in Figure 1.

For bash users, another useful addition is to have Git auto-completion added to your bash shell. This helps you type Git commands faster on the bash command line, by pressing the tab after typing the first few characters of the Git command. The Git auto-completion bash script can be downloaded from <https://github.com/git/git/blob/master/contrib/completion/git-completion.bash>

You can then add the same into your *.bashrc*. For example, the following code can be added to *~/.bashrc*:

```
source ~/.git-completion.bash
```

In addition, the PS1 prompt can be modified to show the branch that you are in, when inside a Git repository. The *.bashrc* can modify the PS1 prompt as follows:

```
PS1='[\u: \w$(__git_ps1 " (%s)")]$ '
```

This results in an output as shown in Figure 2.

This is very handy when you are working on multiple branches. A common error that developers make, i.e., committing code to the wrong branch, can be avoided with this simple step.

Official repository

In Git, all repositories are complete and equal. All developers own their repository. Hence, the official repository just refers to the Git repository identified by the project as 'official' and usually owned by *maintainer*, who is identified by the project team.

All developers clone their individual repositories from this official repository. Note that the *maintainers* themselves may have a cloned repository where they do the development work.

All official builds (release builds, nightly builds, etc) need to be made out of the official repository. Therefore, all code needs to be brought into the official repository ultimately. Thus, code can be considered as committed to the project only after it reaches this official repository from the individual developer's repository.

Cloning the official repository

The official repository can be cloned as follows:

```
$ git clone ssh://userid@ip-addr/path-to-repo [<dir-name>]
```

<dir-name> is optional. If omitted, the same name as in the official repository is created. The Git repository which is cloned is called *origin*.

Branches

Every Git repository has a *master* branch. The *master* branch of a clone is not the same as the *master* branch of the origin. But the *master* branch of a cloned repository tracks the *master* branch of *origin*. It is called a *tracking* branch.

Cloning automatically creates the *master* branch in the cloned repository. Other branches can be created using the following command:

```
$ git branch <branch-name>
```

The branch is local to the repository where it is created. The local branch can be pushed to *origin*.

To move to a specific branch, use the following command:

```
$ git checkout <branch-name>
```

Rules relating to tracking branches

As mentioned above, the *master* branch of a cloned Git repository is a tracking branch. You can track any other